

Sleep vs GPA in College: Predecir el Promedio Académico del Semestre

Emilio López
A01711977@tec.mx

ABSTRACT Este reporte muestra cómo construimos, a mano y con gradiente descendiente, un modelo de regresión lineal múltiple para predecir el GPA del semestre (*term_gpa*) de estudiantes universitarios a partir de datos de sueño y variables demográficas, usando el dataset “*Sleep vs GPA in College*”. Durante la limpieza de datos encontramos y corregimos duplicados, una fuga de información y varias columnas redundantes, y comparamos dos formas de tratar los datos nulos en las variables de carga académica, uno fue imputarlos con la mediana y el otro fue simplemente quitar esas filas. El modelo final, llegó a un R^2 de 0.462 en training y 0.424 en test con la estrategia de imputación, superando en ambos casos a la estrategia de eliminar las filas (R^2 de 0.412 y 0.392). Esto sugiere que el modelo capta una parte de la variabilidad del GPA, sobre todo gracias al GPA previo del estudiante, y que todavía hay espacio para mejorar con modelos no lineales, que es justo lo que se propone como siguiente paso.

1. Introducción

El sueño es uno de los factores que más se ha estudiado en relación con el rendimiento académico. Dormir poco o tener horarios de sueño muy irregulares se ha relacionado consistentemente con menos capacidad de concentración, memoria y desempeño académico en estudiantes universitarios. Pero el desempeño de un estudiante no depende solo de cómo duerme, cosas como su historial académico, cuántos créditos está cursando y su contexto demográfico también importan.

El objetivo de este trabajo es armar, desde cero y sin usar librerías de modelado como `scikit-learn`, un modelo de regresión lineal múltiple que prediga el GPA del semestre (*term_gpa*) de un estudiante a partir de sus patrones de sueño, su historial académico y variables demográficas. La idea no es solo llegar a un modelo que funcione, sino mostrar todo el proceso de limpieza, transformación y modelado de datos, incluyendo decisiones importantes como detectar fugas de información y comparar distintas formas de tratar los datos nulos.

- **student_id**: identificador del estudiante (no único entre universidades).
- **cohort_code**: código de la cohorte (universidad + semestre + estudio).
- **gender, first_generation, underrepresented**: variables demográficas.
- **avg_sleep_hours, daytime_sleep_minutes, sleep_midpoint_minutes, bedtime_variability, nights_tracked_fraction**: variables de sueño.
- **prior_gpa**: GPA acumulado antes del semestre.
- **term_gpa**: GPA del semestre (**variable objetivo**).
- **gpa_change**: diferencia entre *term_gpa* y *prior_gpa*.
- **term_units, term_load_z**: carga académica del semestre (créditos y carga normalizada).

2. Descripción del Dataset

El dataset principal, *college_sleep_and_gpa.csv*, es de Kaggle [1] y tiene 634 filas, cada una es el registro de un estudiante en un semestre en específico. También hay un dataset de referencia, *study_cohort_reference.csv*, que describe las cohortes (*study, university, semester, cohort_code*) y cuántos estudiantes tiene cada una.

2.1.- Features

El dataset tiene 22 columnas, entre las que destacan:

3. Extracción y Limpieza de Datos

3.1.- Duplicados

Al revisar duplicados por *student_id* encontramos varias coincidencias. Pero al investigar esas filas vimos que en realidad eran estudiantes de **distintas universidades** que por casualidad compartían el mismo número de id, no registros duplicados de verdad. Para confirmarlo armamos una llave compuesta, *student_uid* (*cohort_code* + *student_id*), y con esa sí dio cero duplicados, cada fila era efectivamente un estudiante distinto.

3.2.- Valores Nulos

Las columnas *gender*, *first_generation* y *underrepresented* tenían muy pocos nulos (menos del 1%), así que simplemente quitamos esas filas sin que afectara casi nada el tamaño del dataset.

term_units y *term_load_z*, en cambio, tenían bastantes más nulos. Como no estaba claro qué estrategia era mejor, decidimos hacer **dos versiones del dataset** y comparar cuál funcionaba mejor en el modelo final:

- **df_imputado** (626 filas): los nulos de *term_units/term_load_z* se rellenaron con la mediana de cada columna.
- **df_eliminado** (481 filas): se quitaron las filas que tenían nulos en esas columnas.

3.3.- Columnas Redundantes y Fuga de Información

Encontramos y quitamos 9 columnas por ser redundantes, estar derivadas de otras, o directamente filtrar información de la variable objetivo:

- **gpa_change**: es literalmente *term_gpa* - *prior_gpa*, o sea que es una fuga directa de la variable objetivo. Quitarla no era solo por cuestión de limpieza, era necesario para que el modelo no hiciera trampa.
- **study, university, semester**: con tablas de contingencia vimos que tienen un mapeo 1:1 con *cohort_code*, así que no aportaban nada nuevo.
- **avg_sleep_minutes, sleep_midpoint_clock, under_6h_sleep, sleep_bracket**: son la misma información en otro formato (cambio de unidad, de formato, o agrupada en rangos) de columnas que ya teníamos (*avg_sleep_hours* y *sleep_midpoint_minutes*).
- **student_id**: ya lo reemplazamos con *student_uid*, y como identificador puro no aporta nada al modelo.

4. Transformación de Datos

4.1.- Análisis de Correlación

Calculamos la matriz de correlación de Pearson sobre las variables numéricas de *df_imputado* (Figura 1). La variable más correlacionada con *term_gpa* fue, por mucho, *prior_gpa* ($r = 0.638$), seguida de *sleep_midpoint_minutes* ($r = -0.197$), *underrepresented* ($r = -0.173$) y *avg_sleep_hours* ($r = 0.206$).

Esto deja claro que el historial académico previo es el mejor predictor lineal que tenemos, mientras que las variables de sueño, al contrario de lo que pensábamos, aportan algo pero mucho más débil.

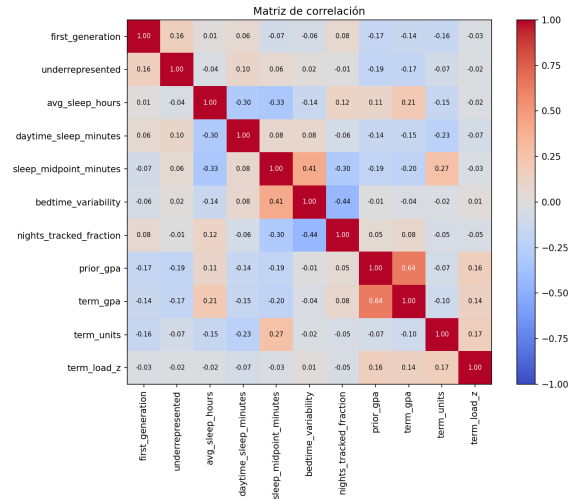


Figure 1: Matriz de correlación entre las variables numéricas del dataset.

La Figura 2 muestra cada predictor numérico contra *term_gpa* por separado, y confirma a simple vista que *prior_gpa* es la única variable con una tendencia lineal clara.

También comparamos *term_gpa* entre cohortes (Figura 3) y sí hay diferencias reales: las medias van de 3.28 a 3.66 según la universidad, así que vale la pena conservar *cohort_code* como predictor. En cambio, la diferencia de *term_gpa* por género fue casi nula (3.449 contra 3.452), así que esa variable probablemente no aporte mucho.

4.2.- Codificación de Variables Categóricas

Las variables categóricas de texto (*gender* y *cohort_code*) se convirtieron a columnas binarias de 0 y 1 con *One-Hot Encoding*, usando **drop_first=True** para que las columnas dummy no queden perfectamente correlacionadas entre sí (si no, la regresión tendría columnas linealmente dependientes).

4.3.- Escalamiento

Antes de entrenar, estandarizamos todas las variables predictoras a media 0 y desviación 1. Importante: la media y desviación usadas para escalar el conjunto de prueba se calcularon **solo con el conjunto de training**, para no filtrar información del test hacia el entrenamiento.

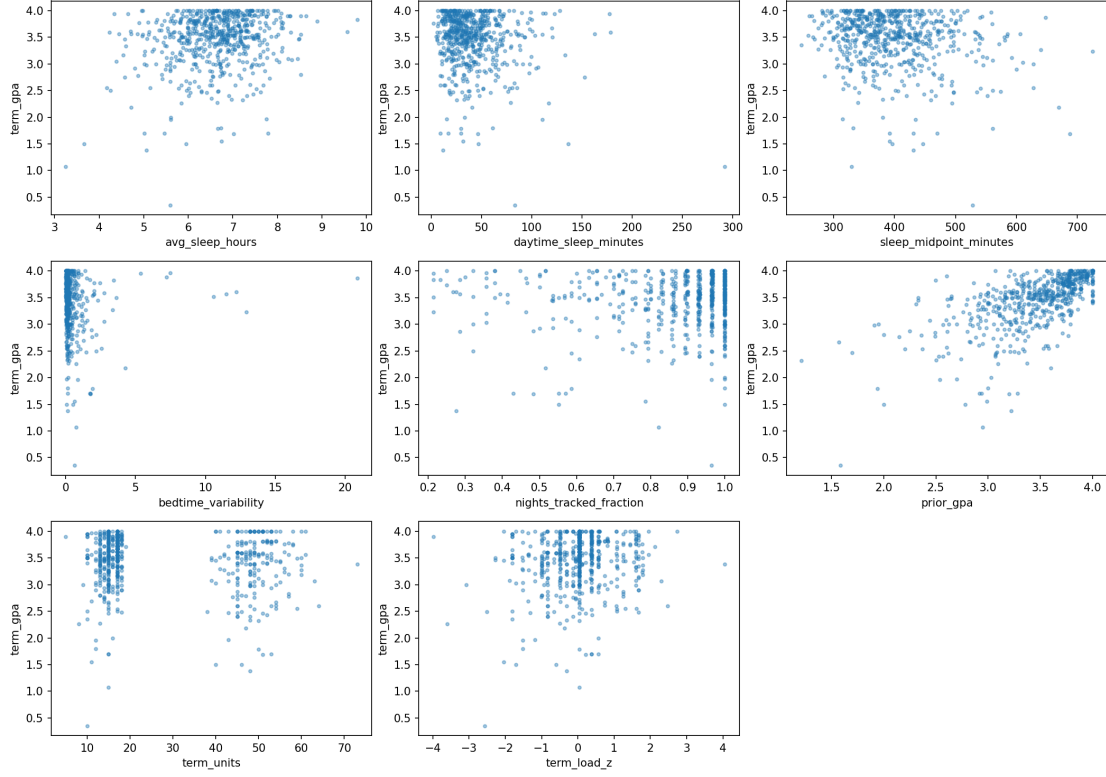


Figure 2: Dispersión de cada variable numérica contra *term_gpa*.

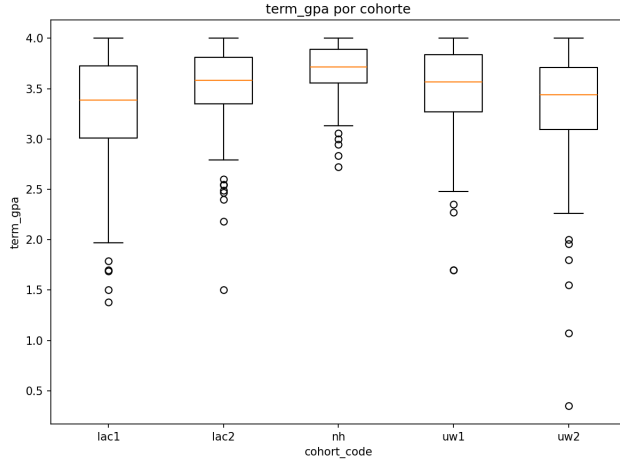


Figure 3: Distribución de *term_gpa* por cohorte.

4.4.- División del Dataset

Dividimos el dataset al azar en 80% para entrenamiento y 20% para prueba, con una semilla fija para que sea reproducible. Esto se hizo por separado tanto para *df_imputado* como para *df_eliminado*.

5. Construcción y Evaluación del Modelo

5.1.- Selección de Features y Variable Objetivo

Usamos todas las columnas que quedaron después de la limpieza (menos *term_gpa*) como predictoras, y *term_gpa* como variable objetivo. Entrenamos y evaluamos el modelo por separado en *df_imputado* y *df_eliminado*, para ver cómo afecta la estrategia de tratar nulos al resultado final.

5.2.- Hipótesis

La hipótesis del modelo es simplemente una combinación lineal de las predictoras más un término de sesgo (*bias*):

$$h(x) = x \cdot \theta + b \quad (1)$$

donde θ es el vector de coeficientes y b es el sesgo, que manejamos como un parámetro aparte en vez de agregar una columna de puros unos.

5.3.- Función de Costo

Usamos el Error Cuadrático Medio (MSE) como función de pérdida:

$$J(\theta, b) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2 \quad (2)$$

5.4.- Descenso de Gradiente

Los parámetros arrancan en cero y se van actualizando, iteración tras iteración, en dirección contraria al gradiente de la función de costo:

$$\theta := \theta - \alpha \cdot \frac{1}{m} X^T (h(X) - y) \quad (3)$$

$$b := b - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i) \quad (4)$$

con una tasa de aprendizaje $\alpha = 0.01$. El entrenamiento para cuando el R^2 de entrenamiento llega a 0.95 o cuando se cumplen 10,000 épocas (al momento de ejecutarlos, ambos modelos corrieron las 10,000 épocas completas sin poder llegar al umbral de 0.95).

5.5.- Evaluación del Modelo

El desempeño se evaluó con R^2 y RMSE:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}, \quad \text{RMSE} = \sqrt{\frac{\sum (y - \hat{y})^2}{m}} \quad (5)$$

6. Visualización y Resultados

La Tabla 1 nos muestra el desempeño final de ambas estrategias de tratamiento de datos nulos.

Dataset	Train R^2	Test R^2	Train RMSE	Test RMSE
df_imputado	0.462	0.424	0.370	0.373
df_eliminado	0.412	0.392	0.396	0.469

Table 1: Comparación de desempeño entre las estrategias de imputación con la mediana y eliminación de las filas.

df_imputado le ganó a df_eliminado en las dos métricas. Lo más notable es que su diferencia entre error de train y test (0.370 a 0.373) es casi nula, mientras que df_eliminado tiene una diferencia más grande (0.396 a 0.469), esto probablemente porque su conjunto de entrenamiento es más chico (481 filas contra 626). Con esta evidencia, **la imputación con la mediana es la estrategia recomendada** por encima de eliminar las filas.

La Figura 4 muestra que los dos modelos convergen bastante rápido (antes de la época 500), y que la ventaja de df_imputado se mantiene estable durante todo el entrenamiento.

La Figura 5 muestra el comportamiento típico de un modelo con R^2 moderado: para GPAs bajos (1.0–2.5) el modelo sobreestima casi siempre, y para GPAs altos (cerca de 4.0) hay bastante dispersión. Esto tiene sentido con un modelo que capta la tendencia general pero le cuesta con los extremos, un indicio de **underfitting** moderado.

7. Siguiendo Pasos: Implementación con Framework

Como siguiente paso, la idea es reimplementar este mismo problema con un framework de *Machine Learning* (por ejemplo, `scikit-learn`) para:

- **Validar la implementación manual:** entrenar un `LinearRegression` (solución cerrada) con los mismos datos y checar que dé coeficientes y R^2 /RMSE parecidos a los que sacamos a mano con descenso de gradiente.
- **Explorar modelos no lineales:** como el R^2 actual (≈ 0.42) sugiere que una relación puramente lineal no está capturando toda la señal, probar modelos como `RandomForestRegressor` o `GradientBoostingRegressor`, que podrían agarrar interacciones no lineales entre sueño, carga académica y GPA previo.
- **Comparar bias y varianza** entre el modelo lineal a mano y los del framework, con el mismo esquema train/test, para saber si el problema es *underfitting* o si el techo de R^2 viene de que los datos de sueño son ruidosos por naturaleza.

Con esta comparación vamos a saber si vale la pena aumentar la complejidad del modelo, o si el modelo lineal que ya tenemos está cerca del límite de lo que se puede explicar con estas variables.

Referencias

- [1] Feng, K. (n.d.). *Sleep vs GPA in College*. Kaggle. <https://www.kaggle.com/datasets/kylefengkfeng209/sleep-vs-gpa-in-college/data>

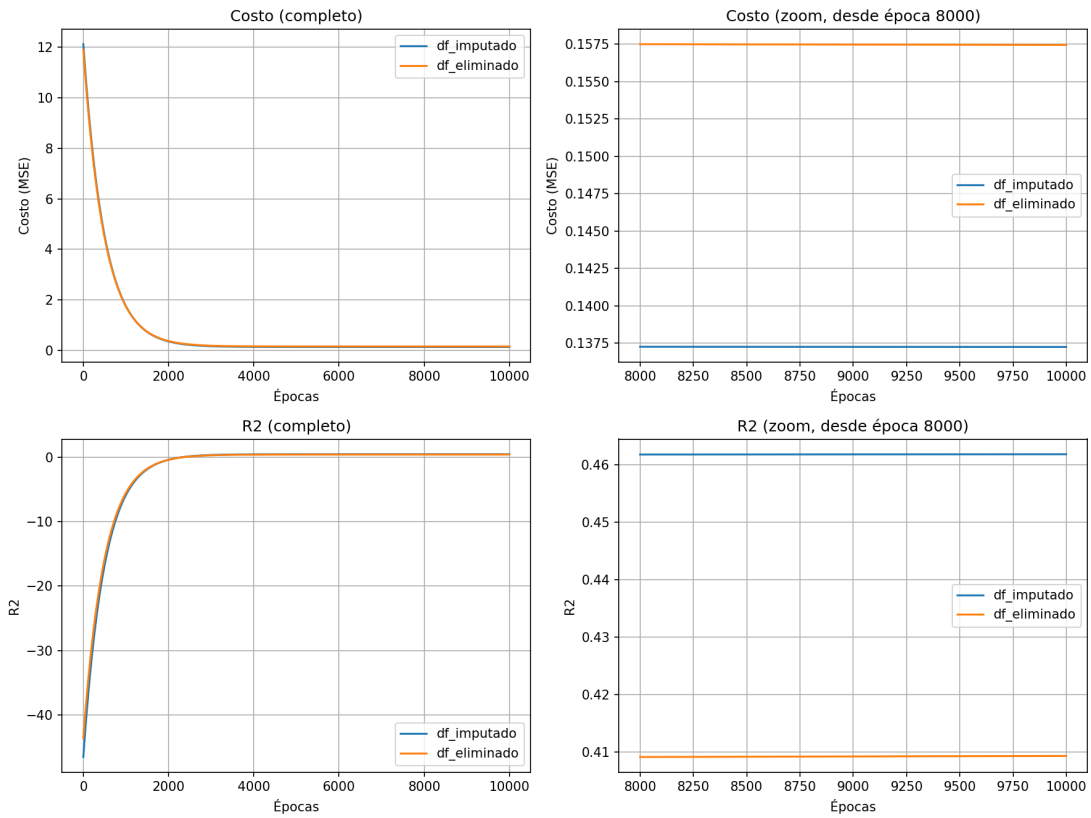


Figure 4: Curvas de costo y R^2 durante el entrenamiento, completas y con zoom en el último 20% de las épocas, para ambas estrategias de imputación.

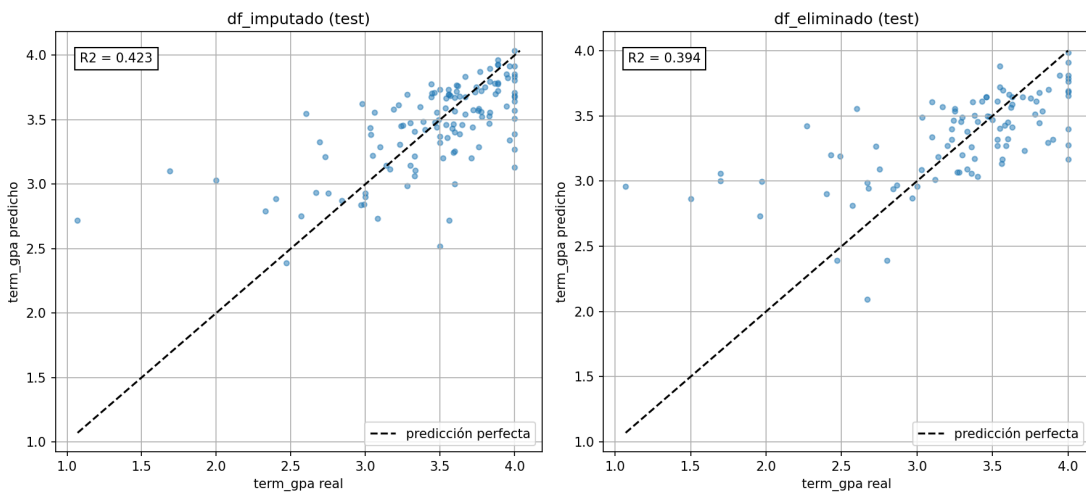


Figure 5: Predicción del modelo contra el valor real de $term_gpa$ sobre el conjunto de prueba, para ambas estrategias.